Chapter-6

Relational Algebra:-

- The relational algebra is a procedural query language.
- It consists of a set of operations that take one or two relations as input and produce a new relation as a result.

fundamental operations:-

- Select operation
- Project "
- Rename "
- Union "
- Intersection "
- set difference "
- Join(/s) "
- Cartesian product "

Select operation:-

- Selects touples that satisfys a given predicate/ condition.

$$\sigma_{\text{Predicate}}(\text{Relation})$$

⊛ ~~select~~ Display all info of books of History genre -

$\sigma_{genre = "History"}(Books)$

⊛ Display all info of books of History genre with rating ≥ 4.0 -

$\sigma_{genre = "History" \wedge rating \geq 4.0}(Books)$

$\geq$
$\leq$
$\neq$
} Allowed

AND → $\wedge$
OR → $\vee$
NOT → $\neg$
} Allowed.

Project operation:-

- Returns its argument relation, with certain attributes left out.

$\Pi_{Attr1, Attr2, ..}(Relation)$

⊛ Display name and rating of all books -

$\Pi_{Book\text{-}name, Rating}(Books)$

⊛ Display name, rating of all the books of "History" genre and rating ≥ 4.0 –

$$\pi_{\text{Book-name, Rating}}\left(\sigma_{\text{Genre}=\text{"History"} \wedge \text{Rating} \geq 4.0}\right)$$

⊞ Rename operation:– → Returns a temporary table

↳ Creates a relation with itself

$$\rho_{x(A_1, A_2, \ldots)}(E)$$

$E \longrightarrow$ Expression

$x \longrightarrow$ New name / Temporary name

$A_i \longrightarrow$ Renamed attributes

⊛ Display Book-name as Name and Price of books in Taka, for last example–

$$\rho_{\text{Temp}(\text{Name}, \text{PriceBDT})}\left(\pi_{\text{Book-name}, \text{Price}*83.3}\left(\sigma_{\text{Genre}=\text{"History"} \wedge \text{Rating} \geq 4.0}(\text{Books})\right)\right)$$

Union operation:-

Π_{Name, No_of_Books} (Publisher)

$\times$ ↺ $\cup$ ∧

Invalid Π_{Name, No_of_Books} (Writers)

Conditions

(i) Same number of attributes.

(ii) Domains of ith attribute of both relation must be same.

Set difference operation:-

- Allows us to find tuples that are in one relation but are not in another relation.

⊛ Find all course taught in the fall 2009 semester but not in Spring 2010 semester.

↳ In Book

$\Pi_{coures\_id}(\sigma_{semester="Fall" \wedge year=2009}(section)) -$

$\Pi_{course\_id}(\sigma_{semester="Spring" \wedge year=2010}(section))$

$\cup$, $-$, $\cap$ } Binary op. σ, π, ρ } Unary op.

Cartesian Product:-

↳ All possible combinations

— Books × Writers = (ID, Book-name, Genre, Writer-ID, Price, Writer-ID, Name, No-of-books)

⊗ Display book name, writer name, rating of all history books

$$\pi_{B\_Name, W\_name, \cancel{genre}\ rating}(\sigma_{Books.writer\_ID = BWriters.Writer\_ID \wedge Genre="History"}(Books \times Writers))$$

(Optimization

$$\pi_{B\_name, W\_name, genre}(\sigma_{Books.W\_ID = Writer.W\_ID}(\sigma_{genre="History}(Books \times Writers))))$$

(Optimization

$$\pi_{B\_Name, W\_Name, genre}(\sigma_{Book.W\_ID = Writers.W\_ID}(\sigma_{Genre="History"}(Books) \times Writers))$$

⊗ emp (ID, Name, Contact, Manager ID)

1	A	-	null
2	B	-	1
3	C	-	2

Display name and corresponding manager name of all the employees.-

$$\Pi_{em.name,\ emp.name}\left(\sigma_{em.manID = emp.ID}\left(\rho_{em}(emp) \times emp\right)\right)$$

⊞ Natural Join:-

X operation
+
σ operation
} ⋈

- Must have a common attribute with same name.

⊗ Display book name and corresponding writer name-

$$\Pi_{book\text{-}name,\ writer\text{-}name}\left(Books \bowtie writers\right)$$

Theta Join:-

$$\bowtie \text{ operation} \; + \; \sigma \text{ operation} \Big\} \Rightarrow \bowtie_\theta$$

⊛ Display bookname and corresponding writer with rating > 4.0

$$\Pi_{\text{book\_name, writer\_name}} (\text{Books} \underset{\text{rating}>4}{\bowtie} \text{Writers})$$

Outer Join:-

- Left outer join → ⟕
- Right " " → ⟖
- Full " " → ⟗

Assignment operation:-

⊛ Display bookname and corresponding writer with rating > 4.0

$$\text{temp1} \leftarrow (\text{Books} \bowtie \text{Writers})$$
$$\text{temp2} \leftarrow \sigma_{\text{rating}>4.0} (\text{temp1})$$
$$\text{temp3} \leftarrow \Pi_{\text{book\_name, writer\_name}} (\text{temp2})$$

Aggregate f^n / Group f^n :-

— Takes a set of values.
— Returns a single value.

Show average rating of the books :-

$$G_{average(rating)}(Books)$$

How many distinct genre?

$$G_{count\text{-}distinct(genre)}(Books)$$

Chapter-7

The entity relationship model:-

- ER model is very useful in mapping the meanings and interaction of the real world enterprises onto a conceptual schema. Because of this usefulness many database design tools draw on concepts from the ER model.
- The ER model employees three basic concepts: entity sets, relationship sets and attributes.

Entity sets:- (Identical to table/relation)

- Entity is a thing or object in the real world that is distinguishable from all other objects. An entity has a set of properties
  - ↳ Attributes
  - Identical to a tuple
- The values for some set of properties may uniquely identify an entity.
  - ↳ Identical to primary key.
- An entity set is a set of entities of the same type that share the same properties or attributes.

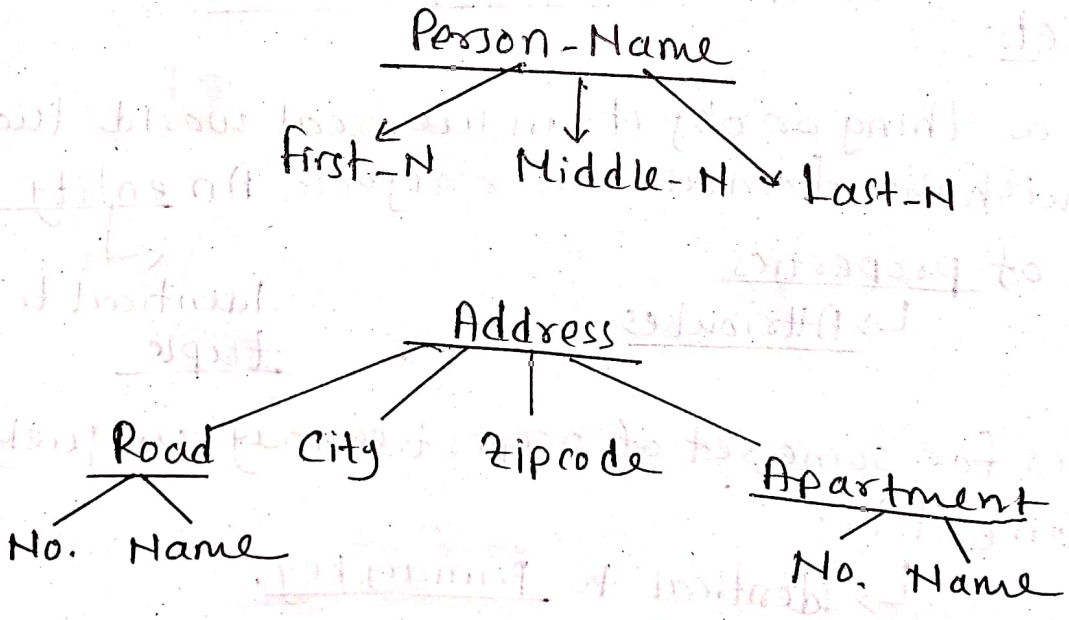
Attributes:-

⊛ Simple and composite:-

Simple
↳ Can't be divided into subparts

Composite
↳ Can be divided into subparts

Ex-



⊛ Single valued and Multivalued:-

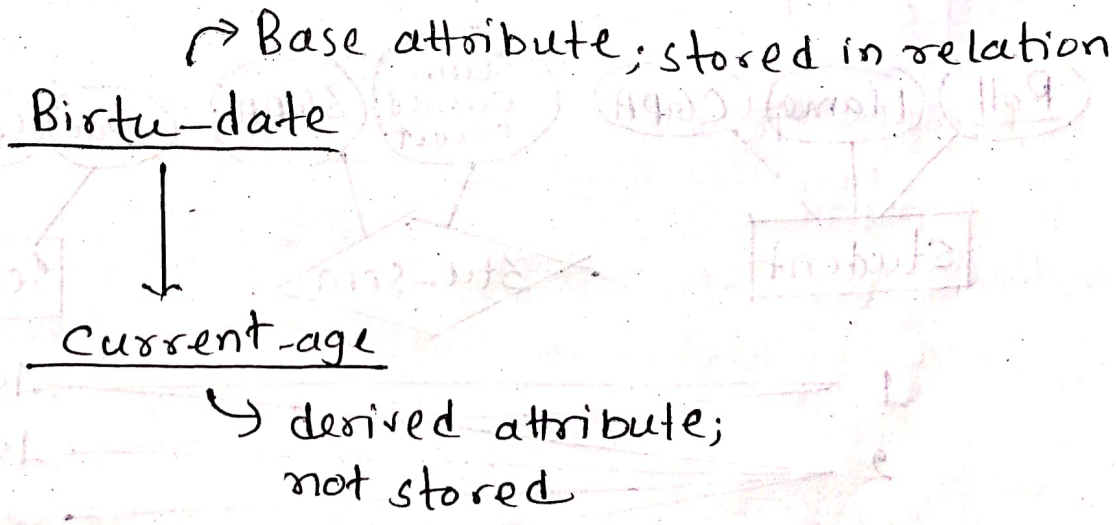
Single valued
↳ Have a single value for a particular entity → { - Roll_No, - Series_No, - Reg_No }

Multivalued
↳ May have zero or more number of values for a particular entity → { -{Mobile_No}, -{Email_ID} }

⊛ Derived attribute:-

- Attributes, that are derived from other related attributes or entities.
- Not stored in database but is computed when required.

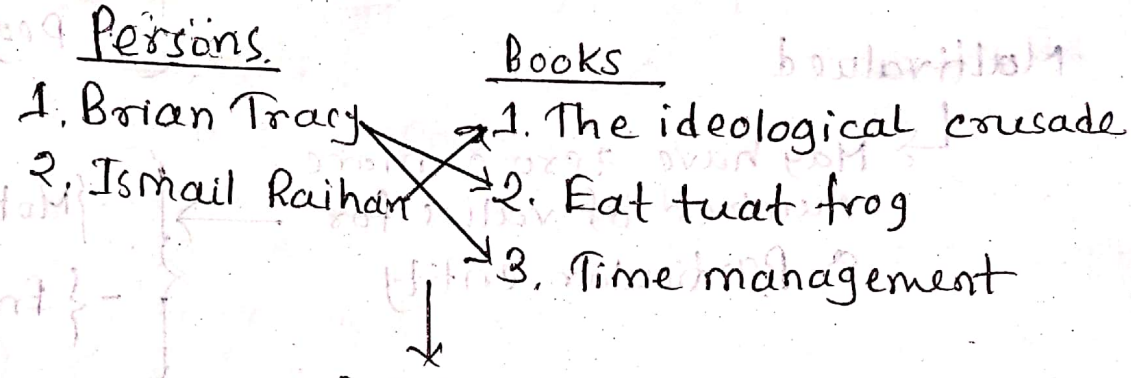
Ex-



Relationship sets:-

- A relationship is an association among several entities.

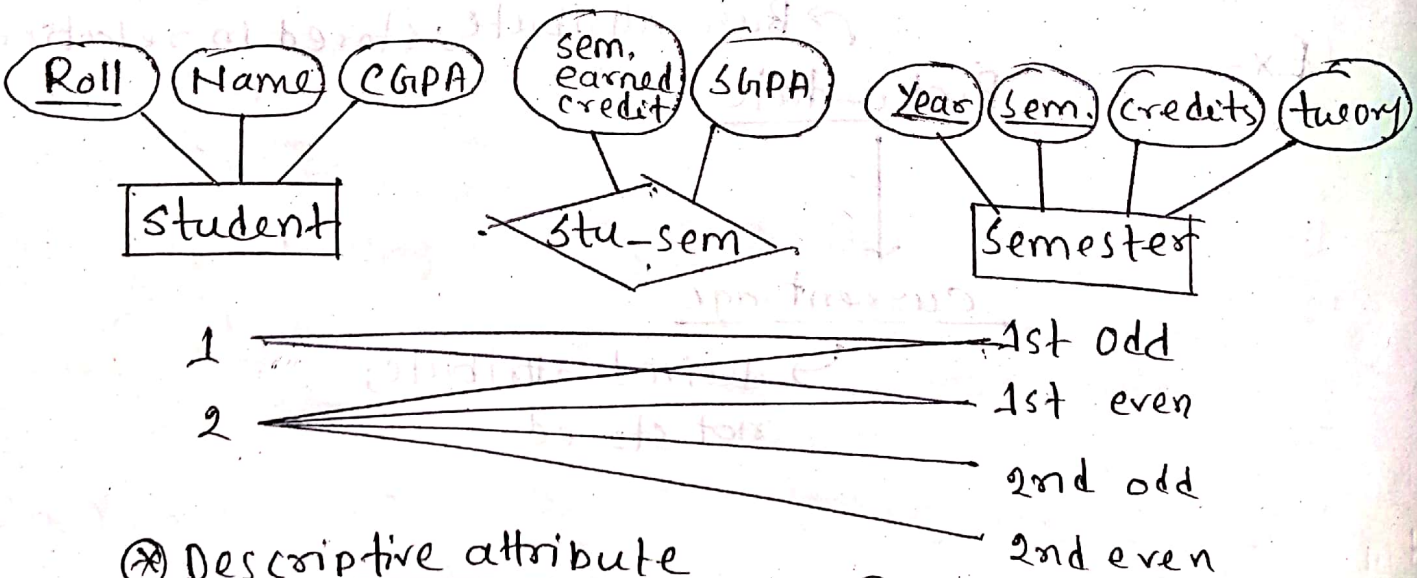
Ex-



set

- A relationship is a set of relationships of the same type.

Ex-



⊛ Descriptive attribute

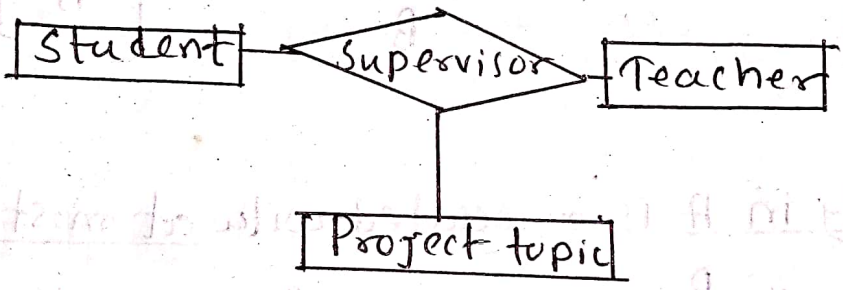
↳ Attributes of a relationship set { - Sem. earned credit, - SGPA }

⊛ Degree of a relationship set

↳ Number of entity sets that participate in a relationship set

Degree-2

↳ Binary relationship



Degree-3

↳ Ternary relationship

⊞ Cardinality ratios/Mapping cardinalities:-

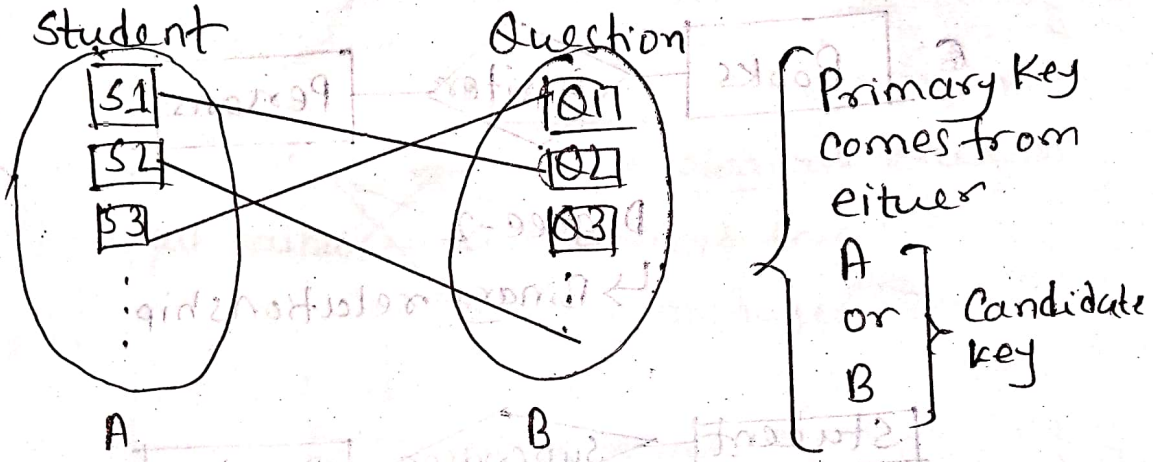
↳ Express the number of entities to which another entity can be associated via a relationship set.

- One to one
- One to many
- Many to one
- Many to many

⊛ One to one:-

Ex-

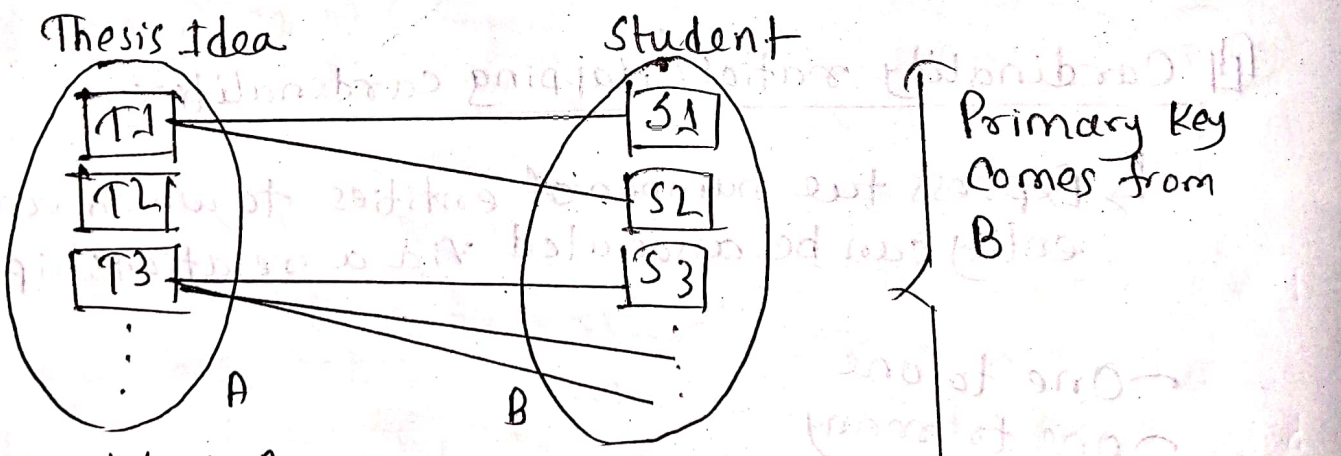
Unique question is assigned to each student-



- An entity in A is associated with at most 1 entity in B
- " " " B " " " " 1 " " A

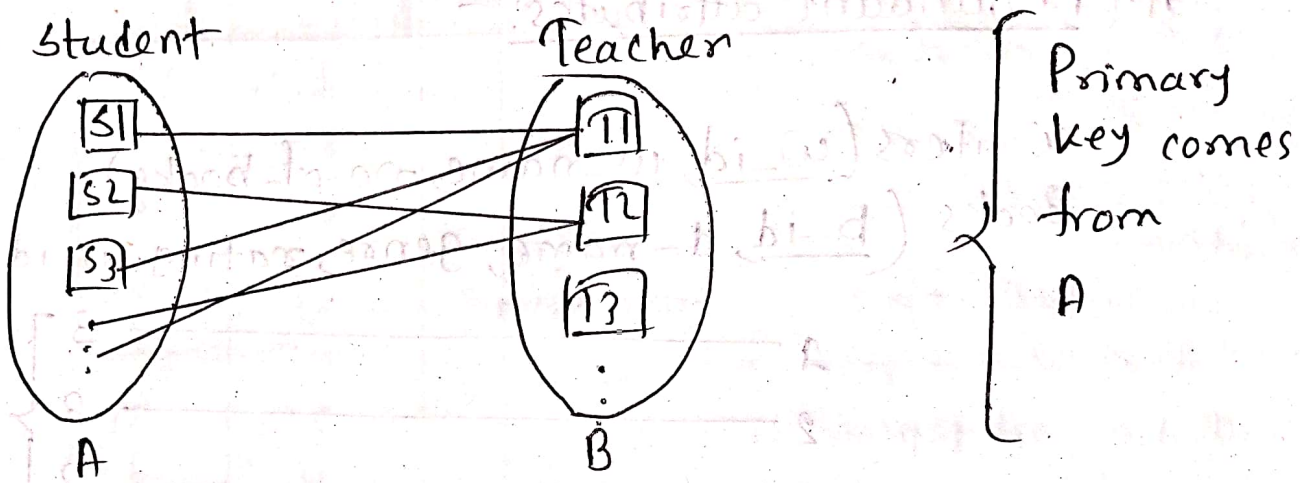
⊛ One to many:-

Ex-



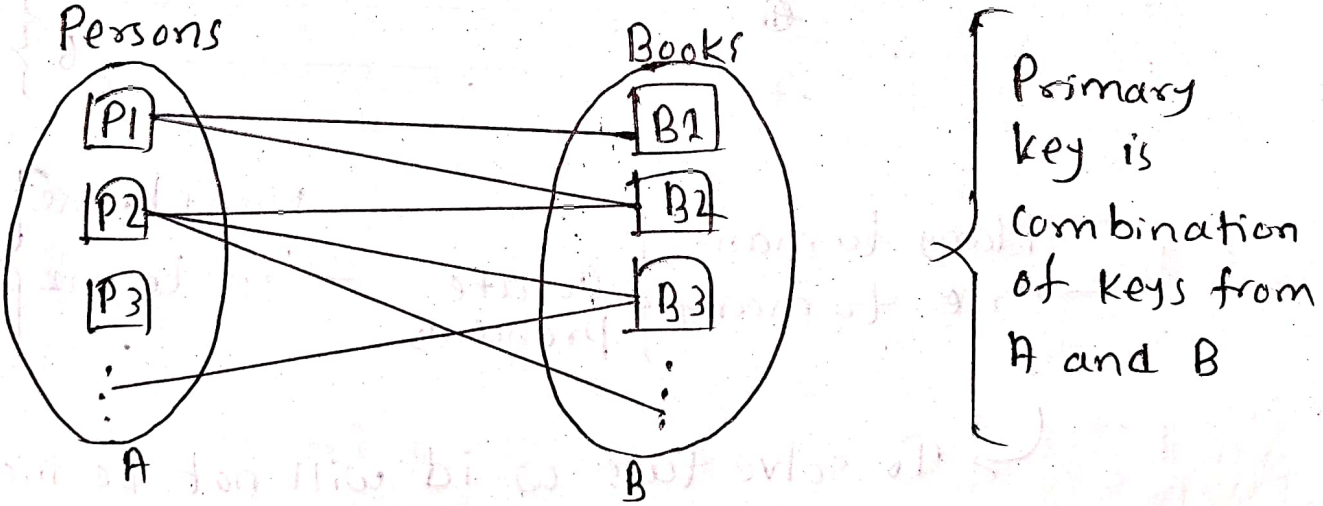
- An entity in A can be associated with 0 or more entities in B
- " " " B " " " " at most 1 " " A

⊛ Many to One:-



- An entity in A is associated with at most 1 entity in B
- " " " B " " " 0 or more " " A

⊛ Many to many:-

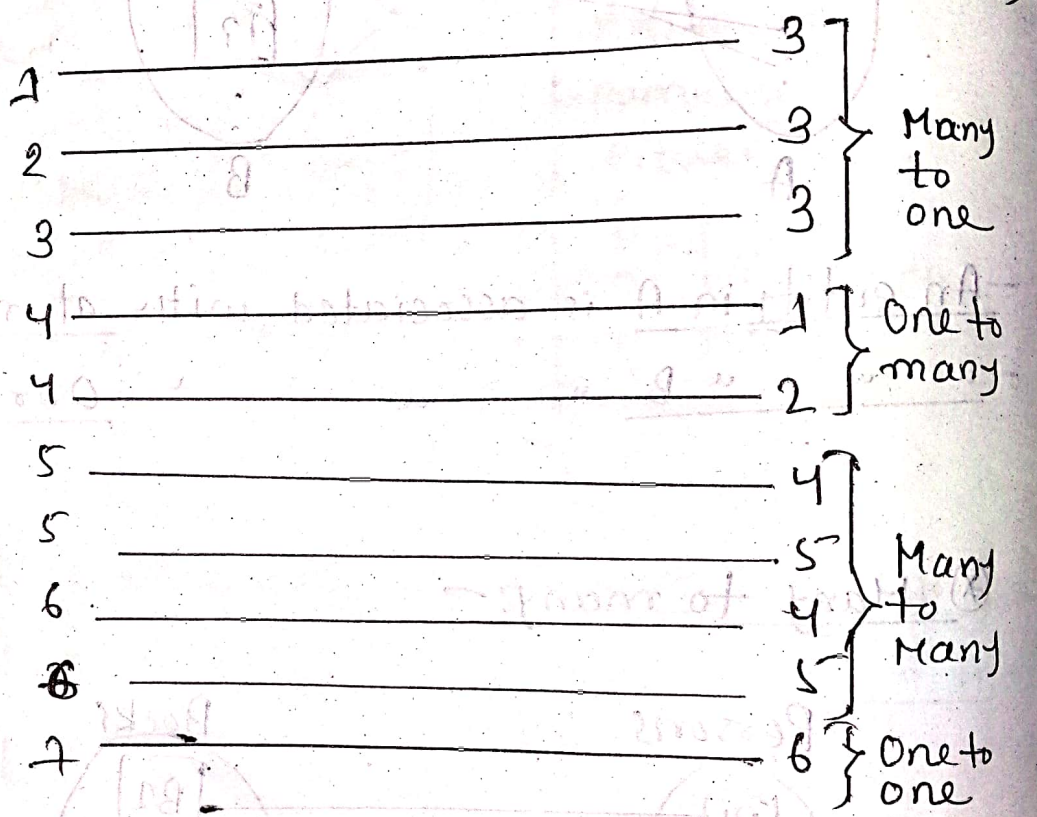


- An entity in A is associated with 0/More in B
- " " " B " " " 0/More " A

Redundant attributes:-

Writers (w_id, w_name, no-of-books)

Books (b_id, b-name, genre, rating, w-id, price)



- Many to many
- One to many

} Create Problems

- Many to one
- One to one

} NO Problem

↳ To solve this, w_id will not be included in that entity set and relationship between b-id and w-id will be recorded in separate table.

book-writer_rel

b_id	w_id
1	3
2	3
3	3
4	1
4	2
5	4
5	5
6	4
6	5
7	6

Primary Key

If there is a many to many relationship, separate table must be included.

ER-Diagram:-

↳ Graphical representation of overall logical structure of a DB

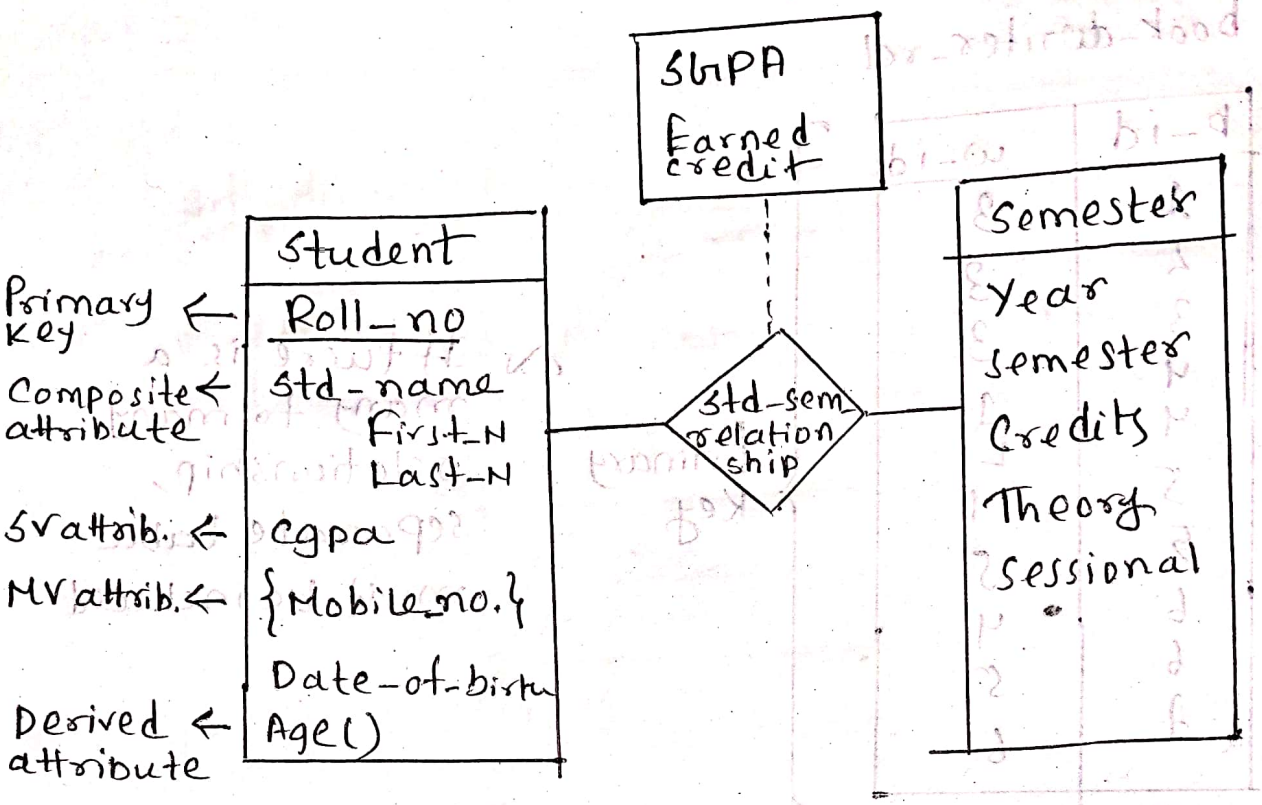
Entity set → Eset_name / Attributes

Relationship set → RS_Name

Relationship set attributes → R_set attributes

ES, RS Link → ———

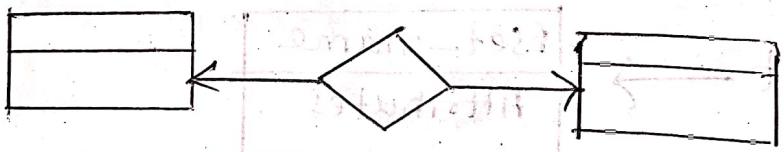
RS, RS attrb Link → - - - - - -



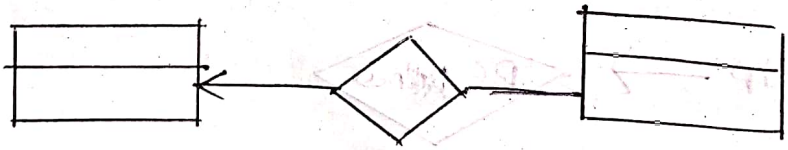
⊛ Cardinality ratio:-

← one
— Many

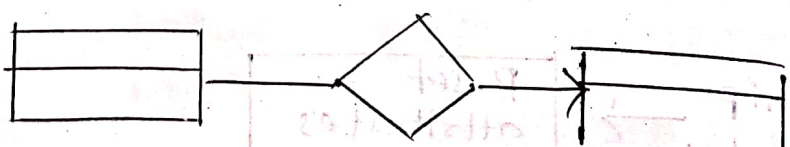
one to one



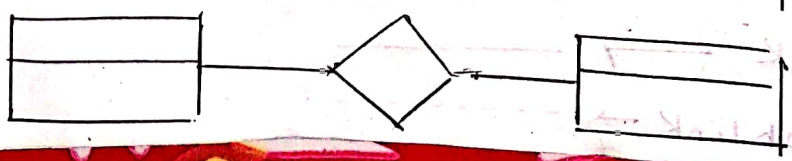
one to many



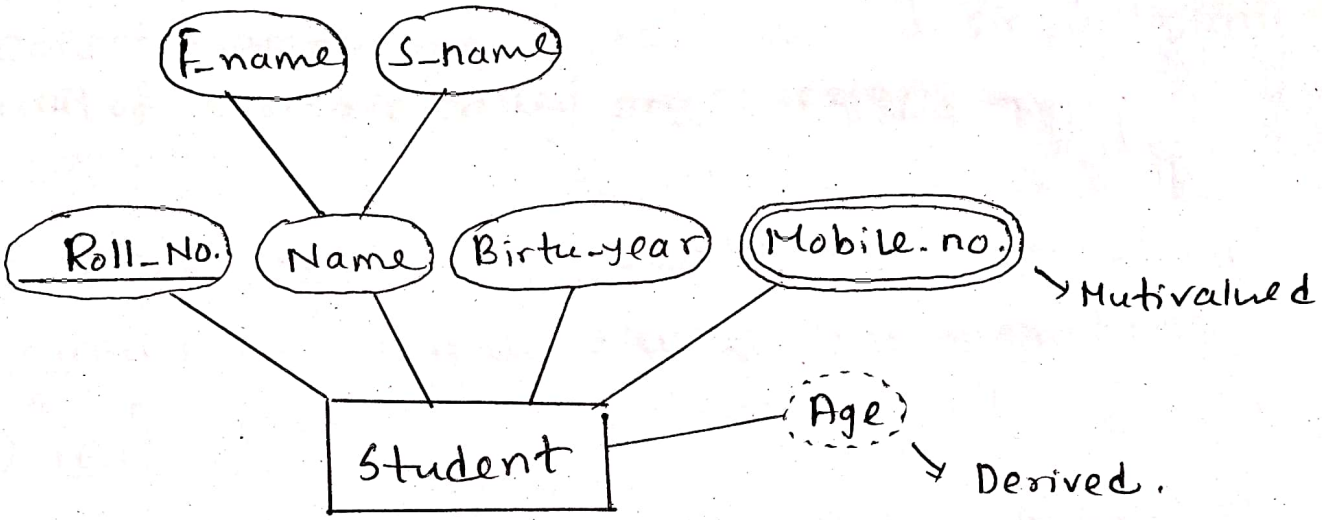
many to one

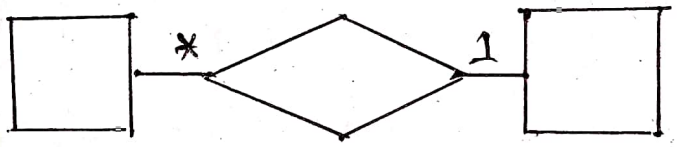


many to many

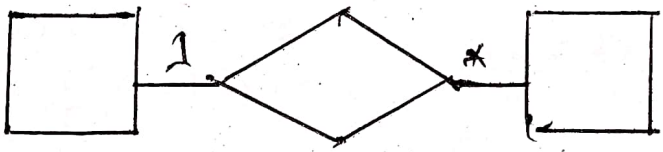


Alternative Representation:-





→ Many to one



→ One to many

* means no limit

Cardinality Limit :-

l ... h
↓ ↓
lower Higher

↷ Many to one

Each student has at most 1 supervisor

Each teacher supervises 0/more student

Chapter-14

Transactions:-

~~- A transaction is a unit of program execution that~~

- collections of operations that form a single logical unit of work are called transactions.

A simple Transaction Model:-

- Transactions access data using two operations

(1) read (X):-

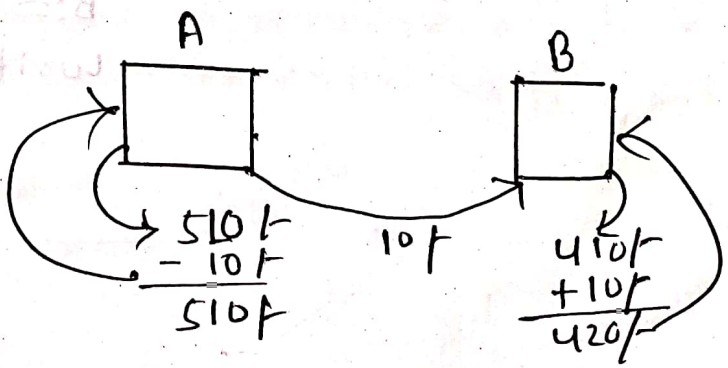
From DB (On disk) → Transfers → A variable X in main memory buffer.

(2) write (X):-

From a variable X in main memory buffer → Transfers → To DB (on disk)

Example-

T_1 : read (A);
A := A - 10;
write (A);
read (B);
B := B + 10;
write (B);



Properties of A DB Transactions:-

ACID properties

- (A) → Atomicity
- (C) → Consistency
- (I) → Isolation
- (D) → Durability

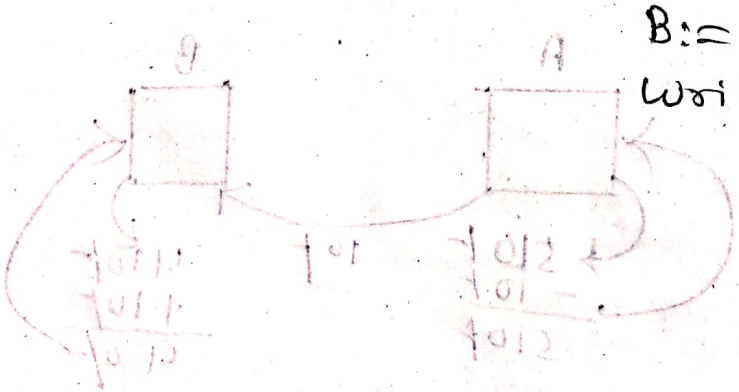
(1) Atomicity:-

↳ Either all operations of the transaction are reflected properly in the database, or none are

T_1 : read(A);
A := A − 10;
write(A);
---------- → Power cut / Sys. crash
read(B);
B := B + 10;
write(B);

↓

The transaction will not occur and DB will be restored.



(2) Consistency:-

↳ DB must be in a consistent state after successful completion of a transaction.

Primary key
Foreign key } Constrains.
Check

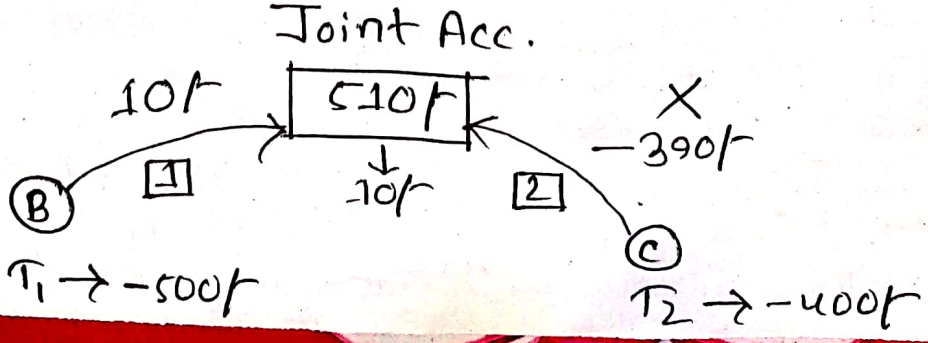
so
↑
A − 500 = −450 ⟶ X

↳ Depends on application

(3) Isolation:-

↳ - Even though multiple transactions may execute concurrently, the system gurantees that for every pair of transactions T_i and T_j, it appears to T_i that either T_j finished execution before T_i started, or T_j started execution after T_i finished.

- Thus, each transaction is unware of other transactions executing concurrently in the system.

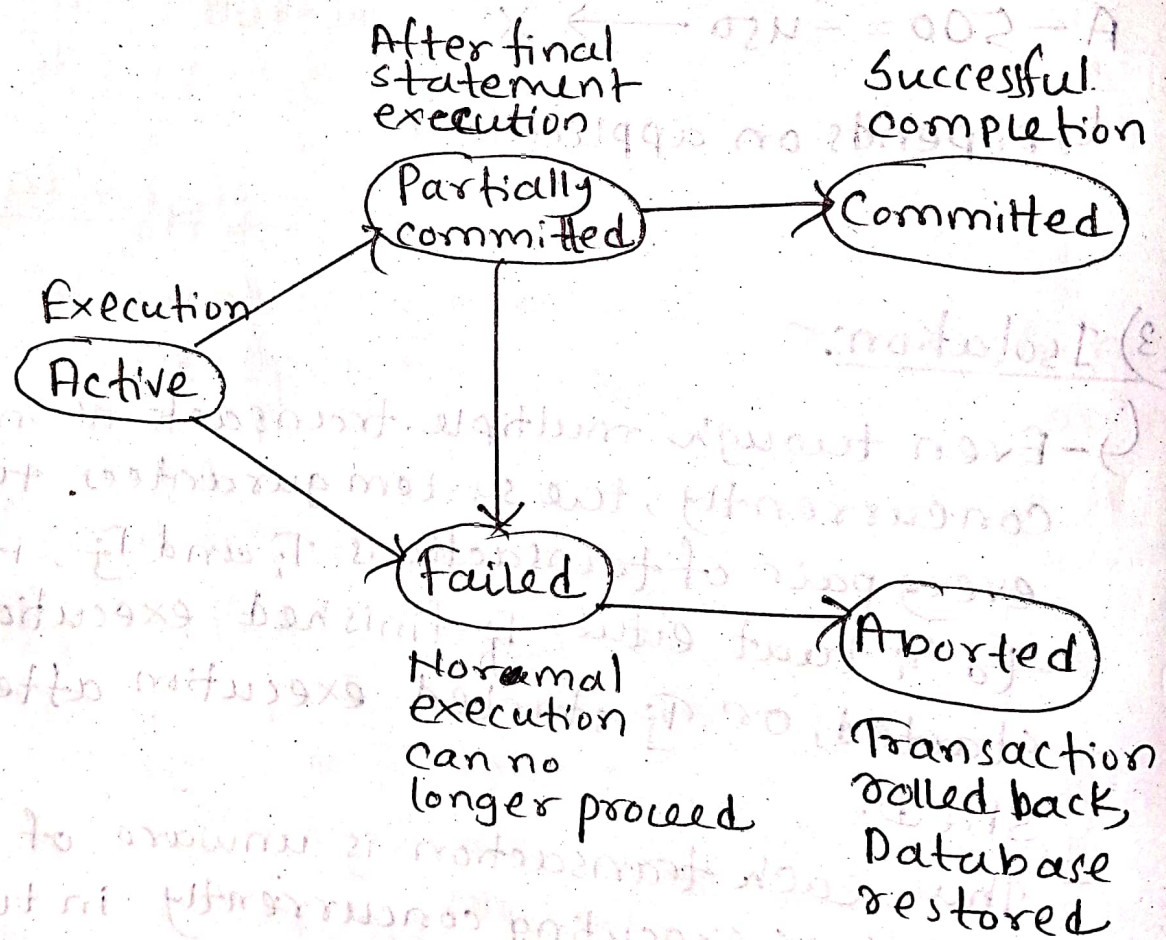


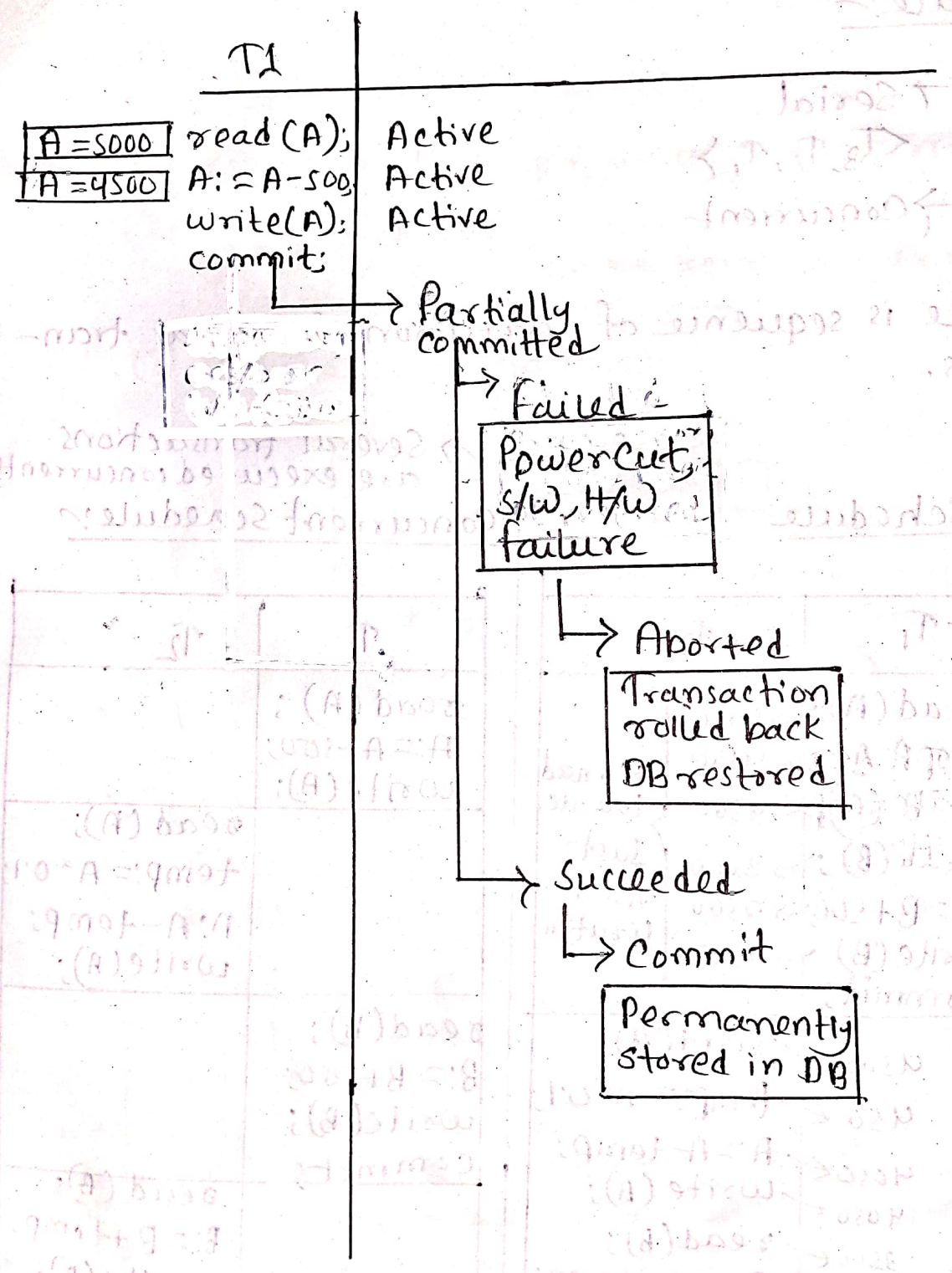
(4) Durability:-

↳ After a transaction completes successfully, the changes it has made to the DB persist, even if there are system failures.

Transaction States:-

State Diagram





TCL → Transaction Control Language

Schedule:-

- Serial
  - $\langle T_3, T_2, T_1 \rangle$
- Concurrent

- Schedule is sequence of execution of several transactions.

↗ Several transactions are executed concurrently

Serial Schedule

T_1	T_2	
read(A);	→ 5000	No need to write. Just visualization
A := A-500;	→ 4500	
write(A);	→ 4500	
read(B);	→ 3000	
B := B+500;	→ 3500	
write(B)	→ 3500	
commit;		
	read(A); ← 4500	
	temp := A*0.1; ← 450	
	A := A-temp; ← 4050	
	write(A); ← 4050	
	read(B); ← 3500	
	B := B+temp; ← 3950	
	write(B); ← 3950	
	commit;	

A+B = 8000

Schedule-1 → A+B = 8000

Concurrent Schedule:-

T_1	T_2
read(A);	
A := A-500;	
write(A);	
	read(A);
	temp := A*0.1;
	A := A-temp;
	write(A);
read(B);	
B := B+500;	
write(B);	
commit;	
	read(B);
	B := B+temp;
	write(B);
	commit;

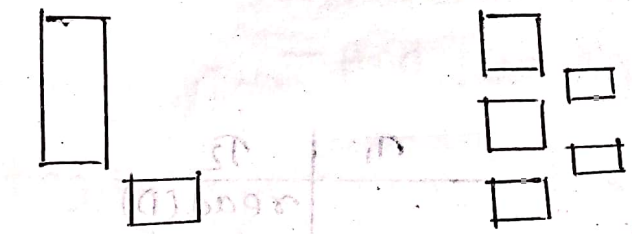
Schedule-2

☐ Advantages of Concurrent:-

(1) Improved throughput and resource utilization.

(2) I/O activities, CPU activities } Can be done in parallel.

(3) Reduce waiting time



☐ Disadvantages:-

— Concurrent schedule may create inconsistency in DB.

Control given to OS

A+B = 8000 → 3000 → 5000

T_1	T_2
read(A); → 3000 A := A-500; → 2500	
	read(A); → 3000 temp := A*0.1; → 300 A := A-temp; → 2700 write(A); → 2700 read(B); → 5000
write(A); → 2500 read(B); → 5000 B := B+500; → 5500 B write(B); → 5500 commit;	
	B := B+temp; → 5300 write(B); → 5300 commit;

Schedule-4 → A+B = 2500+5300 → 7800

Inconsistent → ✗

Two consecutive instructions from different transactions

Case-1

T_1	T_2
read(D)	
	read(D)

T_1	T_2
	read(D)
read(D)	

→ No problem

Case-2

T_1	T_2
read(D)	
	write(D)

T_1	T_2
	read(D)
write(D)	

→ Problem

Case-3

T_1	T_2
write(D)	
	read(D)

T_1	T_2
	write(D)
read(D)	

→ No problem

Case-4

T_1	T_2
write(D)	
	write(D)

T_1	T_2
	write(D)
write(D)	

→ Problem

⊛ Case-2 and 4 creates Inconsistency

→ These are called Conflicting Instructions

→ 2 consecutive instruction from different transactions

- Working on the same data item
- And one of them is write()

- Conflicting instructions are the main cause of inconsistency in DB

⊛ If two consecutive instructions from different transactions

→ Don't conflict

→ Can be swapped

↓

To form an equivalent Schedule

Conflict Equivalent schedule:-

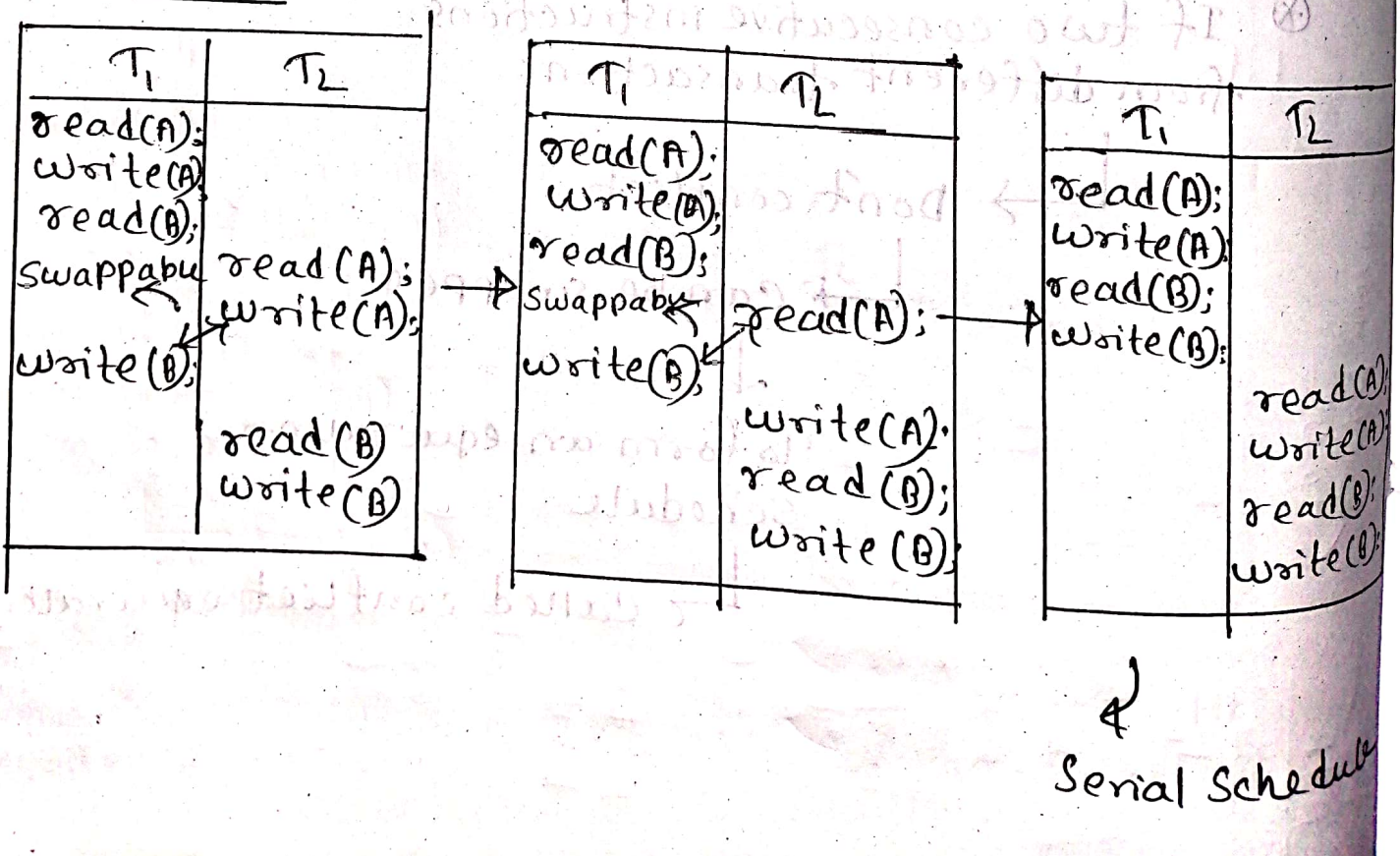
↳ If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, then schedule S' is conflict equivalent to the schedule S.

Serializable schedule:-

↳ A schedule that is conflict equivalent to a serial schedule.

↳ No inconsistency problem

Example:-



Another example:-

T_1	T_2
read(A);	
	read(A);
Swappable	write(A);
	read(B);
write(A);	
read(B);	
write(B);	
	write(B);

→

T_1	T_2
	read(A);
read(A);	
Conflicting	write(A);
	read(B);
write(A);	
read(B);	
write(B);	
	write(B);

↳ Inconsistent

↳ Not serializable

Precedence Graph:-

↳ Directed Graph

When,

(1) T_1 executes read(D) before T_2 executes write(D)
(2) T_1 " write(D) " T_2 " read(D)
(3) T_1 " write(D) " T_2 " write(D)

↳ Conflicting instructions

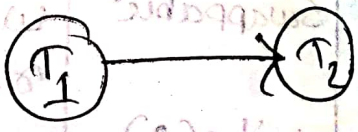
Schedule-3

T_1	T_2
read(A)	
write(A)	
	read(A)
	write(A)
read(B)	
write(B)	
	read(B)
	write(B)

→ Match with (2).

→ Different data. So we don't worry

→ Already an edge between T_1, T_2. So, no new edge

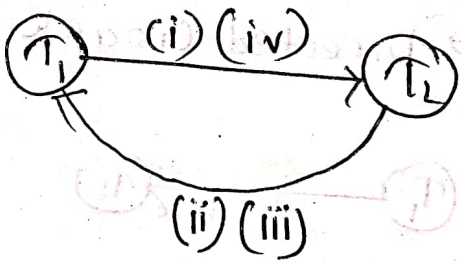


↳ No cycle

↳ Conflict serializable

Schedule-4

T_1	T_2
read(A)	
(i) →	read(A)
	write(A)
(ii) ↓	
write(A)	
read(B)	read(B)
	(iii) ↓
write(B)	
(iv) →	write(B)



↳ Cycle

↳ Not serializable

↳ Inconsistency

Chapter-15

Lock:-

Modes

- Shared Lock → Lock-S(D) → several transactions can hold at a time
  - Read only
  - Cannot be written
- Exclusive Lock → Lock-X(D) → only one transaction can hold at a time. Mutual exclusive.
  - Data item can be read and written

Lock-compatibility Matrix:-

T_i Holds \ T_j Requests	Lock-S(D)	Lock-X(D)
Lock-S(D)	✓	X
Lock-X(D)	X	X

A $\xrightarrow{500}$ B A+B >> Display

T_5	T_6	Consistency Control
Lock-X(A);	---------->	grant-X(A, T_5)
read(A);		
A := A-500;		
write(A);		
Unlock(A);		
	Lock-S(A); --->	grant-S(A, T_6)
	read(A);	
Incon-	Lock-S(B); --->	grant-S(B, T_6)
sistency	read(B);	
	display(A+B);	
Lock-X(B);	---------->	wait
	Unlock(A);	
	Unlock(B);	
Lock-X(B);	---------->	grant-X(B, T_5)
read(B);		
B := B+500;		
write(B);		

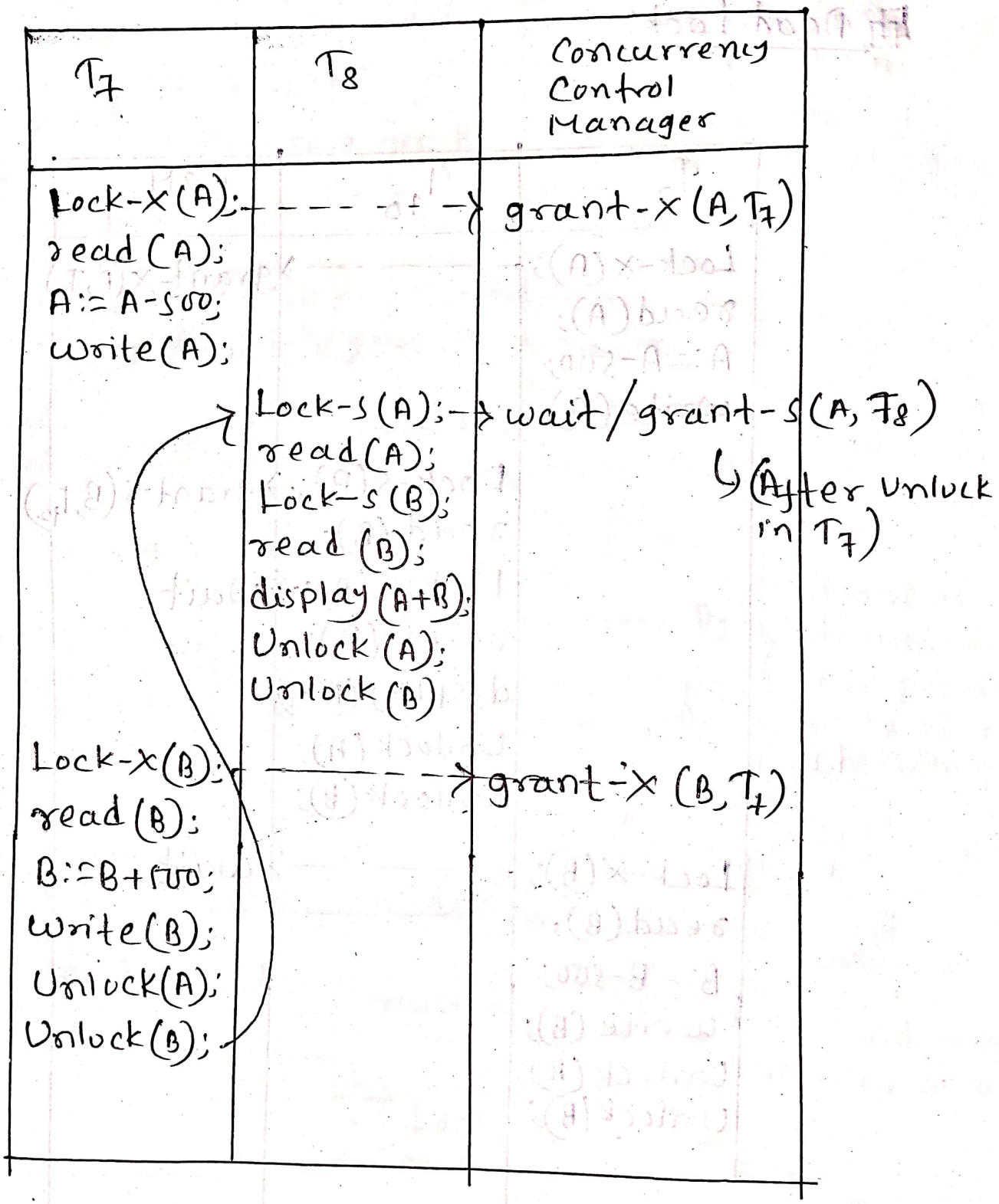
updated ← read(A);

not updated ← read(B);

* Inconsistency occured because A was unlocked early.

* Solution is to unlock at the end.

Solution -



Dead Lock:-

T_9	T_{10}	CCM
Lock-X(A);	———	grant-X(A, T_9)
read(A);		
A := A-500;		
write(A);		
	Lock-S(B);	grant-S(B, T_{10})
	read(B);	
	Lock-S(A);	wait
	read(A);	
	display(A+B);	
	Unlock(A);	
	Unlock(B);	
Lock-X(B);	———	wait
read(B);		
B := B-500;		
write(B);		
Unlock(A);		
Unlock(B);		

T_9
- Holding A
- Requesting for B

T_{10}
- Holding B
- Requesting for A

} Deadlock

Deadlock

↳ T_i is waiting for T_j to release data item D_1
T_j " " " T_i " " " D_2

} None of the transactions can proceed with its normal execution

Concurrent schedule

may create → Inconsistency → Hard to find why inconsistency occured

Soln → Locking

may create → Deadlock → Can be handled

Commit preserves → (A)tomicity

↱ 2PL protocol

Two phase locking protocol:-

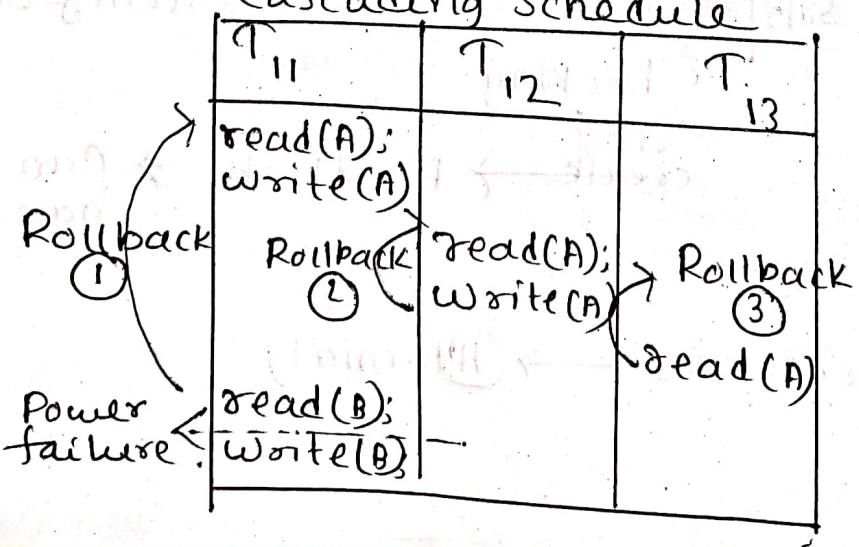
- Growing phase:-
  - Can only obtain lock
  - Can't release any lock
- Shrinking phase:-
  - Can only release lock
  - Can't obtain any new lock

2PL protocol

Advantages
- Ensures serializability (consistency)

Dis. Adv.
- Deadlock may occur
- Cascading rollback may occur

Cascading Schedule



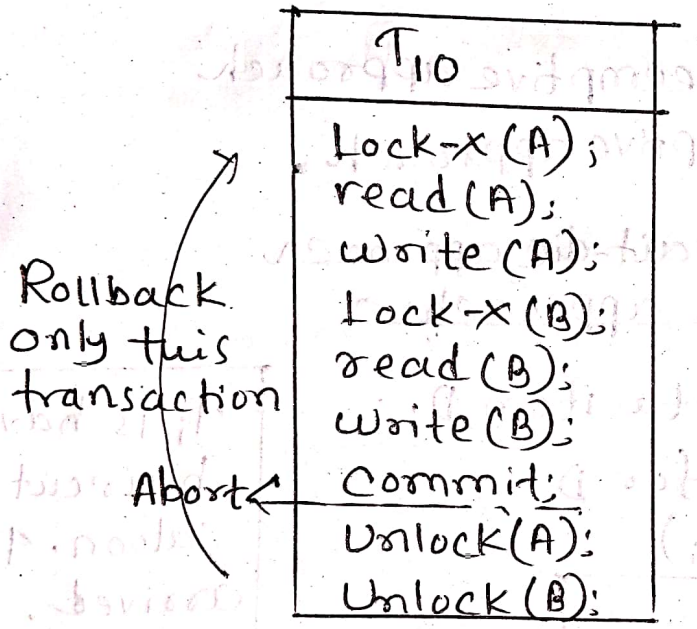
Cascading Rollback

- Failure of a transaction created
  - Series of rollback of several transactions.

Strict Two Phase Locking Protocol:-

→ All the Lock-X(D) are unlocked after commit.

→ Ensures cascadeless schedule



Rigorous Two Phase Locking Protocol:-

→ All the locks are unlocked after commit

→ Ensures isolation

- If other transactions want to use same data current transaction using, we will not allow.
- If the data are different, then no problem.

Dead Lock Handling:-

↳ (1) Deadlock prevention
(2) Deadlock detection and recovery

Deadlock Prevention:-

↳ (1) Non-preemptive approach
(2) Preemptive approach.

↳ wait-die approach

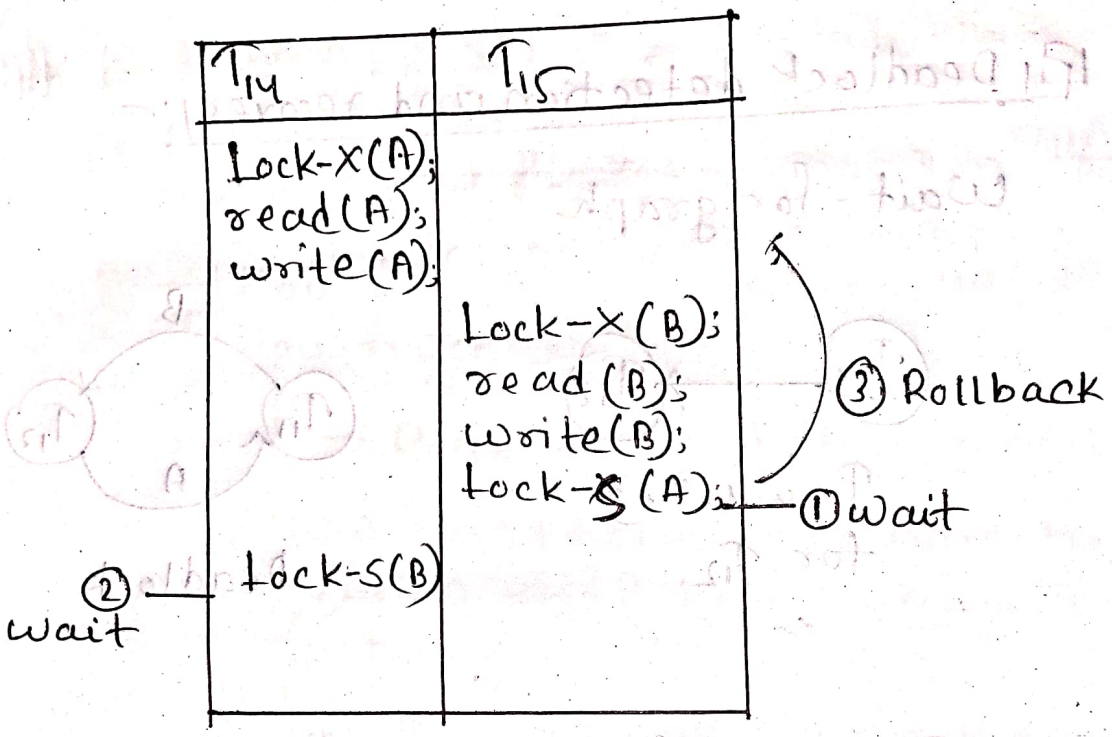
⊛ Non-preemptive approach:-

T_i is holding data item D
T_j is requesting for D

Timestamp(T_j) > Timestamp(T_i)

T_j is older than T_i	T_j will wait
T_j is younger than T_i	T_j will rollback

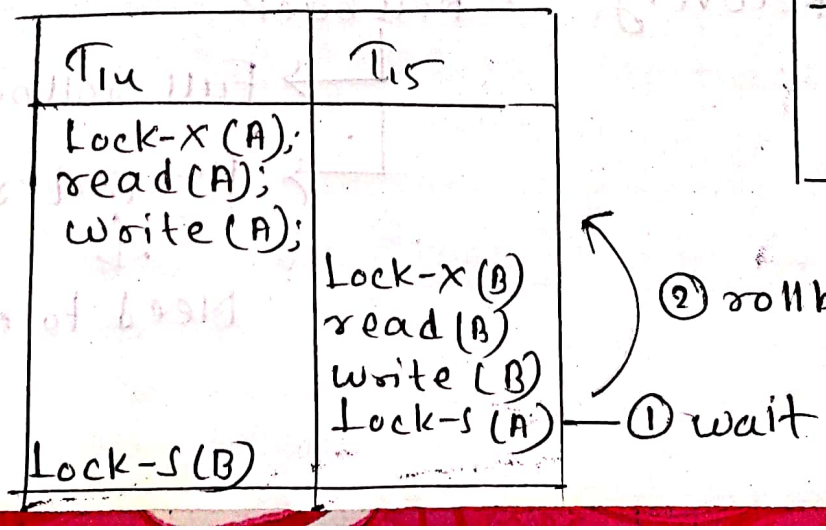
T_i is having haircut at saloon. T_j arrived.
- If T_j is older, T_i: Please wait
- If T_j is young... T_i: Get lost



⊛ Preemptive Approach:

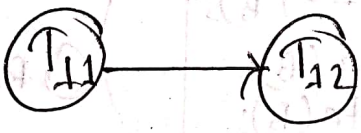
T_j is older than T_i	T_i will roll back
T_j is younger than T_i	T_j will wait

T_i having tea at tong and all sits are filled up. T_j arrives.
- If $T_j < T_i$:
  T_j : :) wait
- If $T_j > T_i$:
  T_i gets chati and kicked out

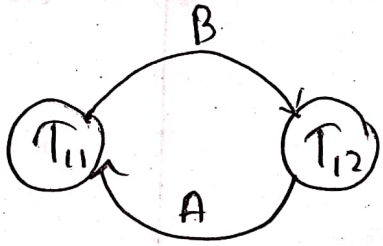


Deadlock detection and recovery:-

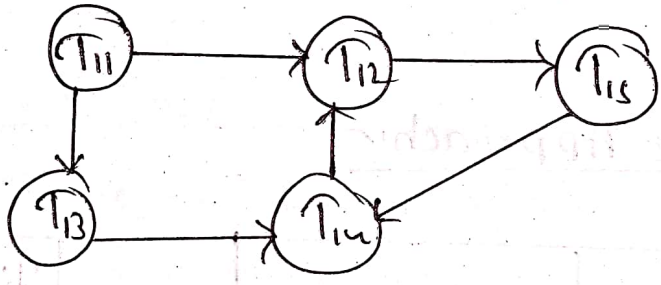
Wait-for graph



T_{11} waiting for T_{12}



Deadlock



$T_{12} \rightarrow T_{15} \rightarrow T_{14} \rightarrow T_{12}$

Deadlock

Recovery → Rollback

- → Full rollback
- → Partial rollback
  - ↓
  - Need to keep track

Deadlock recovery:-

(1) Select a victim depending on minimum cost.

→ How far the transaction has gone? How much left?
  └→ Rollback which has gone less far.

→ How many data items have the transaction used? How many will be used further?
  └→ Rollback which have used more data items.

→ How much dependency? In case of cascading occured.
  └→ Rollback which has less dependency.

(2) Performing roll back

→ Full rollback

→ Partial rollback
  └→ Need to keep track of the transaction.

(3) How many times we will rollback?

→ Everytime, we will not rollback the same transaction again and again. It might create Starvation.

→ We will set a finite number of rollbacks for ~~eaif one is picked as~~ one transaction so that only that transaction will not rollback again and again.

Chapter-8

Normalization:-

→ Technique to organize data in multiple table

→ goal → To minimize redundancy.

Redundancy:-

→ Problems → Anomalies/Abnormality

→ (1) Insertion Anomaly
- Same dept, HoD, OfficeTel needs to be inserted again and again.

(2) Deletion Anomaly
- when we need to delete only tue name, we can't do tuis. The whole tuple is deleted.

(3) Update Anomaly
- If HoD changes, all same name needs to be changed.

Student

RollNo	Name	Dept.	HoD	OfficeTel
1	Rancho	ME	Virus	+910808, +918080
2	Farhan	ME	Virus	+910808
3	Raju	ME	Virus	+910808
4	Acid	CSE		

First Normal Form (1NF):-

(1) Each attribute should be unique.

(2) All values of an attribute should be of same domain.

(3) Each attribute contains single value for each tuple.
(If multivalued attributes are present, separate it)

Student

RollNo	Name	Dept.	HoD

Person_contact_No

Roll	SL	Contact
1	1	-
1	2	-
1	3	-

Second Normal form (2HF):-

→ (1) Satisfies 1HF

(2) No ~~particular~~ partial dependency

Std Marks

Roll	Name	Dept	CourseNo	CourseTitle	Marks	Credit

Partial dependency

Partial dependency

2HF

Student

Roll	Name	Dept

Course

CourseNo	Title	Credits

Marks

Roll	CourseNo	~~Credit~~ Marks